AI OPERATIONS (AIOps)

Module 4 — Model Serving

Course content · one 3-hour lecture · REST APIs, MLflow, Ray Serve, Minikube

1. Overview

Module 4 moves from *building* models (Modules 1–3) to *operating* them: exposing a trained model as a reliable service that other software can call, and releasing new versions without breaking users. The module is taught as a single 3-hour lecture with four topics. Each topic has its own slide deck and its own hands-on Jupyter notebook; every example runs on a laptop CPU.

1.1 Learning outcomes

1. Explain how online, batch and streaming serving differ and choose between them from latency requirements.
2. Build a validated, traceable REST prediction service with FastAPI and test its contract with bad input.
3. Measure latency (p50/p95/p99), throughput and the saturation point of a service with a load generator, and explain the result with Little's Law.
4. Package a model as an MLflow pyfunc with a signature, register versions, manage them with aliases, and serve a registry model.
5. Perform promotion and rollback with each serving layer and explain what must happen (restart, redeploy, rollout) for the change to take effect.
6. Use Ray Serve to scale a model with replicas, split traffic for a canary, autoscale, and observe self-healing.
7. Deploy the service on a 3-node minikube cluster with a Deployment, Service, probes and HPA; execute rolling updates, rollbacks, canary-by-replicas and blue-green switches.
8. Design a post-deployment smoke test that gates a release and triggers automatic rollback.

1.2 Prerequisites

- Modules 1–3: MLflow tracking, Ray Tasks/Actors, Docker (multi-stage builds), Kubernetes Deployments and Services.
- Python 3.10+ with fastapi, uvicorn, scikit-learn, pandas, mlflow>=3, ray[serve], requests, matplotlib, pyyaml.
- Docker, minikube and kubectl for the full cluster exercises (optional for the local-simulation path).

1.3 Scope decisions

What is deliberately NOT covered

BentoML (excluded by the course design). **gRPC / KServe V2 / Triton** are mentioned for comparison only.

GPU serving: everything is CPU-only so every student can run it.

Service meshes (Istio, Linkerd) are named as the way to get exact-percentage canaries on Kubernetes but not exercised.

Block	Topic	Deck	Notebook
1	REST API serving (FastAPI)	Module4_Slides_1_REST_API_Serving	Lecture4a_REST_API_Serving_Exercise

Block	Topic	Deck	Notebook
2	MLflow serving and Registry	Module4_Slides_2_MLflow_Serving	Lecture4b_MLflow_Serving_Exercise
3	Ray Serve	Module4_Slides_3_Ray_Serve	Lecture4c_Ray_Serve_Exercise
4	Minikube / Kubernetes serving	Module4_Slides_4_Minikube_Serving	Lecture4d_Minikube_Serving_Exercise

2. Foundations of model serving

2.1 Why serving is a separate problem

Training optimises a statistical objective once, offline. Serving must satisfy *operational* objectives continuously: low and predictable latency, high availability, safe change. A model file alone has no interface, no input validation, no version identity, no monitoring and no way to roll back. Module 4 adds those, layer by layer.

	Training	Serving
Duration	Minutes to hours	Months
Performance metric	Accuracy, cost per run	p95/p99 latency, availability, error rate (SLOs)
Unit of work	Whole dataset	One request or a small batch
Failure	Rerun the job	User-visible outage

2.2 Serving modes

Mode	Trigger	Latency	Typical tooling	Example
Online	A caller waits	ms – 100s of ms	REST/gRPC server	Churn score during a support call
Batch	A schedule	minutes – hours	Spark job, `mlflow models predict`	Nightly scoring
Streaming	Events	100s of ms – s	Queue consumer + model	Per-transaction fraud flag

The same model artifact can usually serve all three modes; only the wrapper differs. Choosing batch when latency is not needed removes a large amount of operational risk.

2.3 REST versus gRPC

REST with JSON is human-readable, works with any client (`curl`, a browser) and is easy to debug; it pays for that with larger payloads and text parsing. gRPC with Protocol Buffers is compact, typed and supports streaming, at the cost of generated stubs and harder debugging. For small tabular payloads REST usually wins on simplicity; for large tensors or very high QPS between internal services gRPC is often preferred. This module uses REST throughout; the KServe V1-style URL shape (`POST /v1/models/{name}:predict`) is used so the same contract appears in FastAPI, Ray Serve and Kubernetes.

2.4 The running example

A synthetic **customer-churn** classifier with eight features (`tenure_months`, `monthly_charge`, `support_calls`, `late_payments`, `data_usage_gb`, `contract_months`, `num_products`, `age`). `train_models.py` trains two versions deterministically: **v1** (StandardScaler + logistic regression) and **v2** (random forest, 150 trees). On held-out data v1 scores 0.816 and v2 0.840 accuracy. Two versions are essential: they let us practise promotion, canary, shadow, blue-green and rollback. The model bundle stores the model, the **feature column order** and the version string; serving code must never guess column order.

3. REST API serving with FastAPI

3.1 Anatomy of the service

```
MODEL_PATH=data/model_v1.joblib uvicorn rest_app:app --port 8000

GET  /healthz                -> {"status":"ok","model_version":"v1"}
GET  /v1/models/churn        -> metadata (version, features, served_by)
POST /v1/models/churn:predict   -> {"predictions":[0], "probabilities":[0.0579],
                                   "model_version":"v1", "served_by":"<replica>"}
```

- **Load once.** The model is loaded at start-up (module import), never per request.
- **Schema as code.** A pydantic Customer class declares every field and its allowed range; a PredictRequest limits batches to 1–1000 rows. Each Field(ge=..., le=...) encodes a business assumption.
- **Traceability.** Each response carries model_version and served_by (pod or process name). This is what makes canary analysis and incident review possible.
- **Cheap observability.** A middleware adds an X-Process-Time-ms header and counts requests and errors.
- **Safe failure.** Inference exceptions are converted to a generic 500; stack traces never reach the client.

3.2 def versus async def

An async def endpoint runs on the event-loop thread; a CPU-heavy predict() inside it blocks every other request, including health checks. A plain def endpoint is run by FastAPI in a thread pool, which keeps the loop free. Pure-Python work is still limited by the GIL, so capacity is scaled with **worker processes** (uvicorn --workers N) or, later, replicas.

3.3 Contract testing: 4xx versus 5xx

A serving API is a contract. We test it with a table of cases and expect **422** (client error) for each invalid one: negative tenure, missing field, a string where a number is required, an empty list, a wrong top-level key, and age = 500. The model must never be invoked on invalid data. The rule of thumb for clients: 4xx means the client can fix it, so never retry; 5xx or a timeout means retry with back-off.

3.4 Performance: where latency comes from

Measured on the authors' 2-vCPU sandbox (your numbers will differ; the shape will not):

Batch size	Model only (ms)	HTTP total (ms)	Overhead (ms)	HTTP ms / row
1	1.7	7.4	5.7	7.38
10	2.0	7.6	5.6	0.76
100	2.2	9.5	7.3	0.095
500	2.0	15.5	13.5	0.031
1000	2.1	22.4	20.3	0.022

A fixed per-request overhead (JSON parsing, validation, the TCP stack) dominates single-row calls; batching amortises it, but the first row in a batch waits for the last. The overhead grows with batch size here because the pydantic schema validates every row, which is a real cost to budget for.

3.5 Load testing and saturation

load_test.py is a *closed-loop* generator: N concurrent clients each send the next request when the previous one returns. It reports throughput, p50/p95/p99 latency, errors and the number of distinct replicas and model versions that answered. **Always report percentiles, not averages**, and warm the service up first.

Clients	Throughput (req/s)	p50 (ms)	p95 (ms)	p99 (ms)
1	147	6.4	9.3	11.8
2	173	11.0	18.7	23.8
4	157	24.5	41.0	49.1
8	152	50.5	79.7	93.0
16	154	100.6	159.6	219.0

Throughput peaks near two clients and then stays flat while latency rises linearly: the worker is **saturated** and extra clients only queue. This is **Little's Law**, *concurrency ≈ throughput × latency*. The knee of the curve is the service's capacity, and it is the number that autoscalers and SLOs are built on. Running two worker processes lifted throughput to about 276–280 req/s at 4–8 clients (≈ 1.8×) in the same sandbox, even though the load generator shared the two cores with the server.

3.6 Resilient clients

Clients need a timeout on every call, retries only for network errors and 5xx, exponential back-off with a cap, and no retries for 4xx. The notebook's safe_predict implements this; its dead-port test shows 4 attempts and ≈ 1.4 s of back-off before it gives up.

4. Serving with MLflow

4.1 From model file to governed asset

Module 1 used MLflow Tracking to compare runs. Serving adds three pieces: the **pyfunc** interface, a **signature**, and the **Model Registry**. Everything the server needs is stored with the model — wrapper code, artifacts, dependency list and the input/output schema — so mlflow models serve needs no application code.

4.2 pyfunc and signatures

python_function is MLflow's universal serving flavor: predict(DataFrame) → DataFrame. A custom PythonModel loads the joblib bundle in load_context and returns a DataFrame with churn_probability, churn and model_version, so served answers stay traceable. The **signature** declares typed input columns (eight doubles) and outputs. Inputs are cast to float64 so JSON integers and floats are both accepted. input_example is stored and validated against the signature at logging time.

```
with mlflow.start_run(run_name="churn-v2"):
    info = mlflow.pyfunc.log_model(
        name="model", python_model=ChurnPyfunc(), artifacts={"bundle": "data/model_v2.joblib"},
        signature=sig, input_example=X.head(2),
        pip_requirements=["scikit-learn", "pandas", "joblib", "numpy"],
        registered_model_name="churn-model")      # creates the next version
```

4.3 Registry: versions and aliases

Versions are immutable and numbered. **Aliases** (champion, challenger) are movable names. A client or server refers to models:/churn-model@champion; promotion is moving the alias to a new version and rollback is moving it back. Aliases replace the older, deprecated stage model (Staging/Production). The Registry needs a database-backed tracking store, so every shell must export MLFLOW_TRACKING_URI=sqlite:///mlflow.db.

4.4 Serving and the contract

```
mlflow models serve -m "models:/churn-model@champion" -p 5001 --env-manager local
POST /invocations   {"dataframe_records":[{...8 features...}]}
GET  /ping    GET /version
```

Accepted input formats include dataframe_records, dataframe_split, TF-Serving style instances, and inputs tensors. Schema enforcement rejects missing columns and non-numeric values with HTTP 400, but **silently accepts extra columns and applies no range checks** (age = 500 is accepted). A signature checks types and presence, not business rules. When rules or custom routes are needed, wrap the pyfunc inside your own FastAPI app.

4.5 Promotion, rollback and the restart

Moving the alias is a metadata operation and is instantaneous, but a **running** mlflow models serve process keeps answering with the version it resolved at start-up. Promotion therefore needs a restart (in Kubernetes, a rolling update). Rollback is the same two steps with the old version. Practical pitfall found while preparing this lecture: a restart can appear to succeed against the **old** process if the old process has not yet released the port, so verify model_version in a response rather than trusting /ping.

4.6 Batch scoring and containers

- mlflow models predict -m models:/churn-model@champion -i batch_in.json -t json -o out.json scores a file without a server.
- mlflow models build-docker -m … -n churn-mlflow:champion produces an image with the pinned environment and the same server (needs a Docker daemon; not executed in the authoring sandbox).

4.7 MLflow serve versus hand-written FastAPI

Dimension	mlflow models serve	FastAPI
Time to first endpoint	Minutes	Hours
Validation	Types and presence	Any rule
Custom routes and auth	No (wrap it)	Full control
Measured, 1 client	≈ 73 req/s, p50 12 ms	≈ 147 req/s, p50 6 ms

4.8 Safe release practices

Promotion gate. Evaluate champion and challenger on *fresh* data neither has seen. In our run the champion scored 0.828 and the challenger 0.8375, a gain of 0.95 points; a one-point bar blocks promotion. The train_accuracy MLflow logged for v2 (0.89) was in-sample and misleading. **Shadow test.** Send identical requests to both servers, answer clients from the champion only, and compare. We measured an 8.5 % label disagreement and p95 latency of 16.6 ms (champion) versus 37.0 ms (challenger) — information available before any user is affected.

5. Serving with Ray Serve

5.1 Architecture

Ray Serve is a serving library on Ray. A **deployment** is a Python class (or function) replicated as Ray actors; an **HTTP proxy** accepts requests; an ingress **router** deployment can contain routing logic; a **controller** creates, replaces and scales replicas to match the declared state — the same reconcile-loop idea Kubernetes uses. Replicas reserve CPU from the Ray cluster, and **fractional CPUs** (0.25) allow a laptop to host many replicas.

```
@serve.deployment(num_replicas=2, ray_actor_options={"num_cpus": 0.25},
                  max_ongoing_requests=16, health_check_period_s=5)
class ChurnModel:
    def __init__(self, path, extra_work_ms=0.0): ...
    async def __call__(self, rows): ...
    async def check_health(self): ...      # raise -> replica is restarted
```

5.2 Composition and canary

A router deployment receives HTTP requests (FastAPI ingress), draws random() < canary_weight and forwards to the v1 or v2 **DeploymentHandle**. With weight 0.2, 300 requests gave 240 to v1 and 60 to v2, each served by two distinct replicas. With 300 samples the standard error is about 2.3 percentage points, which is why real canaries run until enough traffic has been observed.

5.3 Scaling: an experiment must test what it claims

The first scaling experiment was flat (48.6, 51.0, 53.0 req/s for 1, 2, 4 replicas): on a two-core machine the HTTP proxy, router and load generator saturated the CPUs before the tiny model did. The experiment was redesigned, not the data: extra_work_ms=80 adds a blocking 80 ms step to each replica (a stand-in for a larger network; a sleep, so it costs no CPU). Then throughput rose from 9.3 to 16.2 req/s (1.74×) with two replicas and 19.8 (2.13×) with four, bending as another stage became the bottleneck (a second full run gave 10.2, 16.5 and 20.8 req/s — same shape, different numbers). **Scale the bottleneck, and say what you simulated.**

5.4 Autoscaling and self-healing

autoscaling_config (min/max replicas, target_ongoing_requests, up/down delays) lets the controller follow load. With 12 concurrent clients for 25 s, replicas went from 1 to 4 and, after the idle period, back to 1. Up-fast/down-slow delays damp oscillation. When a replica actor was killed under traffic, 87 requests succeeded with 0 errors and the replica count returned to two. One practical gotcha: in one run, replacing an autoscaled application directly with another serve.run was followed by a stalled controller (a KeyError in the controller log); calling serve.delete(app) first and waiting a few seconds avoided it.

5.5 Release engineering

Stage	Weight to v2	v1 share	v2 share	Errors
canary 20 %	0.2	79 %	21 %	0
canary 50 %	0.5	47 %	53 %	0

Stage	Weight to v2	v1 share	v2 share	Errors
promote 100 %	1.0	0 %	100 %	0
rollback 0 %	0.0	100 %	0 %	0

Changing the router's argument redeploys only the router; model replicas keep their loaded weights, so there is no cold start and rollback is as fast as promotion. **Composition** is ordinary Python: an Ensemble deployment awaited both model handles in parallel (asyncio.gather) and averaged probabilities (0.1029, between v1's 0.0579 and v2's 0.148).

6. Serving on Kubernetes with Minikube

6.1 Design

Three minikube nodes run as containers on the laptop (`--nodes 3 --cpus 2 --memory 2200`, about 6 GB and 6 vCPUs in total). Two Deployments (`churn-v1` ×3, `churn-v2` ×1) carry the label `app=churn`; a Service selecting only that label balances across all four pods, giving about 75 % / 25 % **canary by replica ratio**. A second Service, `churn-bg`, selects a single version for **blue-green**. An HPA scales `churn-v1` between 2 and 6 pods.

6.2 Images and manifests

`Dockerfile.serve` is a multi-stage build that bakes `model_v*.joblib` into the image, so every pod on every node has the model without shared storage. `minikube image build` builds inside the cluster's runtime (no registry), hence `imagePullPolicy: Never`. Key manifest fields:

- `strategy: RollingUpdate {maxSurge: 1, maxUnavailable: 0}` — start a new pod, wait for readiness, then remove an old one; capacity never dips.
- `topologySpreadConstraints` on `kubernetes.io/hostname` — spread replicas over nodes (`ScheduleAnyway` = best effort).
- Downward API `POD_NAME` — exposes the pod name as `served_by`.
- `resources.requests` drive scheduling and the HPA's utilization maths; `limits` cause throttling.

6.3 Readiness versus liveness

Readiness asks *may I send you traffic?* A failing pod is removed from the Service endpoints but not restarted. **Liveness** asks *are you stuck?* A repeatedly failing pod is restarted. Anti-patterns: a liveness probe that depends on a downstream database (cascading restarts), identical aggressive settings for both, a probe path the app does not serve (CrashLoopBackOff), and an `async def` CPU-bound handler that makes the probe time out under load.

6.4 Operating the service

```
kubectl apply -f k8s/deployment-v1.yaml -f k8s/deployment-v2.yaml -f k8s/service.yaml
kubectl rollout status deploy/churn-v1
kubectl scale deploy/churn-v1 --replicas=6
kubectl set image deploy/churn-v1 server=churn-serve:v2     # rolling update
kubectl rollout undo deploy/churn-v1                       # rollback
kubectl patch svc churn-bg -p '{"spec":{"selector":{"app":"churn","version":"v2"}}}'   # blue-green
```

Strategy	Traffic to new version	Rollback	Extra cost	Granularity
Rolling update	Gradual replacement	Minutes (rollout undo)	+1 surge pod	Pods
Canary (replica ratio)	≈ new / (new + old)	Seconds (scale v2 to 0)	Small	Coarse, min 1/N
Blue-green	0 → 100 % at once	Instant (selector back)	2× capacity	All or nothing

Strategy	Traffic to new version	Rollback	Extra cost	Granularity
Shadow	Copy only	n/a	Extra compute	Per request
Weighted canary (mesh / Ray router)	Exact %	Seconds	Mesh / router	Per request

6.5 Horizontal Pod Autoscaler

```
desiredReplicas = ceil( currentReplicas × currentUtilization / targetUtilization )
utilization = CPU usage / CPU REQUEST       (can exceed 100 %)
sync period 15 s · 10 % tolerance · scale up fast · scale down to the highest
recommendation of the last stabilization window (60 s in our manifest)
```

Example: 2 pods at 100 % of request with a 50 % target → ceil(2 × 100/50) = 4. In a Python simulation (40 req/s per pod at 100 % of request, target 50 %, min 2, max 6) the replica count lags a traffic ramp, caps at maxReplicas while utilization stays above target (8 pods would be needed to keep 300 req/s below 100 %), and holds capacity through the stabilization window after traffic drops. This is a model of the algorithm, not a measurement of a real HPA.

6.6 Release safety nets

smoke_test.py runs after a rollout: health check, expected model version, 30 predictions inside a latency bound, and an invalid request that must return 422. Exit code 0/1 drives the pipeline: kubectl rollout status ... || kubectl rollout undo ... then the smoke test, with kubectl rollout undo on failure. Two gates protect users: Kubernetes readiness (does it start?) and our smoke test (is it correct and fast?).

What was and was not tested while preparing this module

Executed and verified: the FastAPI service, MLflow registration/serving/aliases/batch CLI, Ray Serve (canary, scaling, autoscaling, kill, ensemble), the local pod/endpoint simulation, the HPA simulator, manifest linting, the smoke test, and all four notebooks end to end.

Not executed (no Docker daemon in the authoring sandbox): minikube start, image builds, kubectl apply/rollout/undo, NodePort access, the real HPA with metrics-server, and mlflow models build-docker. Treat those commands as a runbook to validate on a student machine.

7. Cross-cutting themes

Idea	FastAPI	MLflow serve	Ray Serve	Kubernetes
Who writes the API	You	MLflow (from signature)	You (Python)	Your container
Versioning	model_version field	Registry aliases	Per deployment	Image tags, ReplicaSets
Scale-out	--workers	Manual	Replicas + autoscaling	Replicas + HPA, multi-node
Canary	—	Shadow (2 servers)	Router weight	Replica ratio / mesh

Idea	FastAPI	MLflow serve	Ray Serve	Kubernetes
Rollback	Redeploy	Alias + restart	Redeploy router	rollout undo, selector
Self-healing	Process manager	—	Controller	ReplicaSet + probes

The same operational ideas — health, versions, traceability, canary, rollback, autoscaling — recur at every layer. Learning them once, in the simplest layer, makes the others legible.

8. Glossary

Term	Meaning
SLO	Service-level objective, e.g. p95 latency ≤ 80 ms.
p95 / p99	Latency below which 95 % / 99 % of requests complete.
Saturation	Point where added load only increases queueing delay.
Little's Law	Concurrency ≈ throughput × latency.
pyfunc	MLflow's universal Python model interface.
Signature	Typed input/output schema stored with a model.
Alias	Movable name for a registry version (e.g. champion).
Replica	One running copy of a deployment (Ray actor or Kubernetes pod).
Canary	Sending a small share of traffic to a new version.
Shadow	Copying traffic to a new version and ignoring its answers.
Blue-green	Two full stacks with an atomic pointer switch.
Readiness / liveness	Probes that gate traffic / trigger restarts.
HPA	Horizontal Pod Autoscaler.
Smoke test	Fast post-deployment check that gates a release.

9. Further reading

- Official documentation: FastAPI, MLflow (Models, Model Registry, Deployment), Ray Serve, Kubernetes (Deployments, Services, Probes, HPA), minikube.
- Google SRE Book — chapters on service-level objectives and handling overload.
- Chip Huyen, *Designing Machine Learning Systems* — deployment and monitoring chapters.

Version notes: material was prepared with FastAPI 0.143, MLflow 3.17, Ray 2.59, scikit-learn 1.9 and Python 3.13. Pin versions in your environment; MLflow and Ray in particular change quickly.